

Multi-Agent Chess Arena

A Minimax Agent vs. an Alpha-Beta Pruning Agent with Heuristic Evaluation
23CSE401 – Fundamentals of Artificial Intelligence: Case Study

Name: Nikhil Sanjay Roll No.: BL.EN.U4CSE23239

Abstract

We build a chess arena in which two game-playing agents compete: a depth-limited **Minimax** agent and an **Alpha-Beta** agent. Both use the same heuristic evaluation (material plus piece-square tables), so every difference between them comes from the search technique. On five test positions at depths 1–4, alpha-beta returned exactly the minimax value in every run while visiting far fewer positions. At depth 4 from the starting position it searched 3,109 nodes against 206,604 for minimax. Two working cases (a mate and a fork) and three edge cases (the horizon effect, a stalemate trap and bad move ordering) show how the agents behave in normal and difficult positions. In a 24-game arena, equal-depth agents play at the same strength, but when alpha-beta spends its savings on one extra ply of search, at less than twice minimax's node budget, it scores 8.5–3.5.

1 Problem Domain and Statement

Domain. Chess: a two-player, turn-based board game on an 8×8 board. White and Black each start with 16 pieces and alternate moves. A player wins by checkmating the opponent's king. A game can also end in a draw (stalemate, threefold repetition, insufficient material, 50-move rule).

Agents. Two software agents play each other in an *arena* (a game loop that asks the side to move for a move, applies it and checks for the end of the game):

- **Minimax agent (MM):** looks ahead d plies and considers *every* sequence of legal moves. It scores the positions at depth d with a heuristic and backs the values up the tree.
- **Alpha-Beta agent (AB):** the same look-ahead and the same heuristic, but it *prunes* branches that provably cannot change the decision. It also orders moves (captures first) so that pruning happens early.

Goals. Each agent tries to maximise its own game outcome (win > draw > loss). Chess is zero-sum, so one agent's gain is exactly the other's loss.

Interaction. The agents share one board and move alternately. Every move changes the options of the opponent, and each agent has to predict the other's reply. This is what makes the problem multi-agent and adversarial.

Problem statement. Given a chess position, choose a move that is best for the side to move, assuming the opponent also plays its best replies, within a limited amount of computation. Then measure (i) whether the two search techniques choose equally good moves, (ii) how much computation each needs and (iii) how this translates into results when the agents play each other.

2 PEAS Description

Performance measure	Game result (win = 1, draw = $\frac{1}{2}$, loss = 0) over many games; material balance; quality of each chosen move (its minimax value); computational cost per move (nodes searched, time).
Environment	The 8×8 board with its pieces, the opponent agent, and the rules of chess (legal moves, check, checkmate, stalemate, repetition, promotion, castling, en passant). Rules are provided by the <code>python-chess</code> library.
Actuators	Selecting and playing one legal move on the agent's turn (a piece move, capture, castling, en-passant capture or promotion).
Sensors	The complete board state (FEN): piece placement, side to move, castling rights, en-passant square and move counters; the list of legal moves; whether the position is check, checkmate or a draw.

3 Environment Properties

Property	Value	Justification
Observability	Fully observable	Both agents see the whole board and every piece. Nothing is hidden (unlike poker).
Determinism	Deterministic	A move always has exactly the stated effect; there is no chance element. The only uncertainty is the opponent's choice, which is handled by adversarial search, not probabilities.
Episodic / sequential	Sequential	Every move affects all later positions. A pawn grabbed now can lose the queen two moves later (case E1).
Static / dynamic	Static	The board does not change while an agent is thinking. (With a chess clock it would be <i>semi-dynamic</i> : the position stays fixed but the remaining time shrinks.)
Discrete / continuous	Discrete	64 squares, a finite set of legal moves, and discrete turns.
Single / multi-agent	Multi-agent	Two agents whose objectives are exactly opposed (competitive, zero-sum).
Known / unknown	Known	The rules, i.e. the transition model, are fully known to both agents.

4 Type of Agent

Both players are **utility-based, model-based agents with adversarial look-ahead**:

- *Model-based*: the agent uses the rules of chess (the transition model $\text{RESULT}(s, a)$) to simulate moves internally before playing one.
- *Utility-based*: a goal test (“is it checkmate?”) is almost never reachable within the search horizon, so the agent needs a *utility estimate* of non-terminal positions. That estimate is the heuristic evaluation function of Section 5. The agent picks the move with the best backed-up utility.

A *simple reflex* agent (e.g. “capture the most valuable piece”) is not enough. In case E2 it captures a knight and stalemates a completely won position. A *goal-based* agent without a utility estimate cannot rank moves when no mate is in sight.

Cooperative or competitive? The agents are **purely competitive**. Chess is a two-player *zero-sum* game: $U_{\text{White}} = -U_{\text{Black}}$. There is no joint objective and no communication. Each agent therefore models the other as an adversary that always chooses the reply worst for it, which is exactly the assumption behind minimax: White is MAX, Black is MIN. Within each agent the search is a solo computation, and the two agents never cooperate.

5 Heuristic Evaluation Function

Both agents use one evaluation, from White’s point of view in centipawns (100 = one pawn):

$$\text{eval}(s) = \sum_{p \in \text{White}} (V(p) + \text{PST}_p(\text{sq})) - \sum_{p \in \text{Black}} (V(p) + \text{PST}_p(\text{sq}))$$

with piece values $V = \text{P } 100, \text{N } 320, \text{B } 330, \text{R } 500, \text{Q } 900$, and Michniewski’s piece-square tables PST. These reward centralised knights, advanced pawns and a castled king, and penalise knights on the rim. Terminal positions are scored exactly: checkmate = $\pm(100000 - \text{ply})$, so a faster mate is preferred, while stalemate and insufficient material score 0. The evaluation is cheap ($O(\text{pieces})$) but ignores king safety, mobility and threats. Section 7 shows the consequences.

6 Searching Technique

6.1 Formulation as a search problem

- **State:** a chess position (board, side to move, castling/en-passant rights).
- **Initial state:** the current position. **Players:** MAX = White, MIN = Black.
- **Actions:** the legal moves (on average $b \approx 35$ in the middlegame, 20 at the start).
- **Terminal test:** checkmate, stalemate, insufficient material.
- **Utility / cut-off:** at depth d the heuristic replaces the true game value.

The state space of chess has about 10^{43} – 10^{47} positions and the game tree about 10^{120} nodes (Shannon), so the search must be depth-limited.

6.2 Minimax

$$\text{MM}(s, d) = \begin{cases} \text{eval}(s) & d = 0 \text{ or terminal} \\ \max_a \text{MM}(\text{RESULT}(s, a), d - 1) & \text{White to move} \\ \min_a \text{MM}(\text{RESULT}(s, a), d - 1) & \text{Black to move} \end{cases}$$

The recursion is a depth-first traversal of the whole tree to depth d . The agent plays the root move with the best backed-up value.

6.3 Alpha-beta pruning

Alpha-beta computes the same $\text{MM}(s, d)$ but carries two bounds: α , the best value MAX is already guaranteed on the current path, and β , the best value MIN is already guaranteed. At a MIN node, as soon as one reply gives a value $\leq \alpha$, MAX will never enter this node, because it already has something better. The remaining replies are therefore skipped (a *cut-off*). The same holds symmetrically at MAX nodes. Pruning never changes the result. It only avoids work, and our experiments **assert** this on every run. The amount of pruning depends on the order in which moves are tried. Our AB agent sorts moves by **MVV-LVA** (captures of the Most Valuable Victim by the Least Valuable Attacker first), then promotions and checks.

6.4 State space, search tree and sequence of explored states

To make the tree readable, Figure 1 uses a small king-and-pawn endgame (8/8/8/4k3/8/8/4P3/4K3 w) at depth 2, and both searches use the same (generator) move order. White has 6 moves and Black 6–8 replies. The sequence explored by alpha-beta is:

1. **Kf2:** all 8 replies searched; the minimum is 100, so $\alpha = 100$.
2. **Kd2:** the first reply Kf6 already scores $100 \leq \alpha$. MIN can hold White to 100 here, so Kd2 cannot beat Kf2, and the other **7 replies are pruned**.
3. **Kf1:** every reply is $\geq 110 > \alpha$, so all are searched; the minimum is 110 and α becomes 110.
4. **Kd1:** first reply $100 \leq \alpha$, so **7 replies are pruned**.

5. **e3**: minimum 120, so $\alpha = 120$ (the new best move).
 6. **e4**: the reply ...Kxe4 wins the pawn (value $0 \leq \alpha$), so the last reply is pruned.
- Minimax evaluates all 51 positions and alpha-beta 36. Both pick **e3** with value 120.

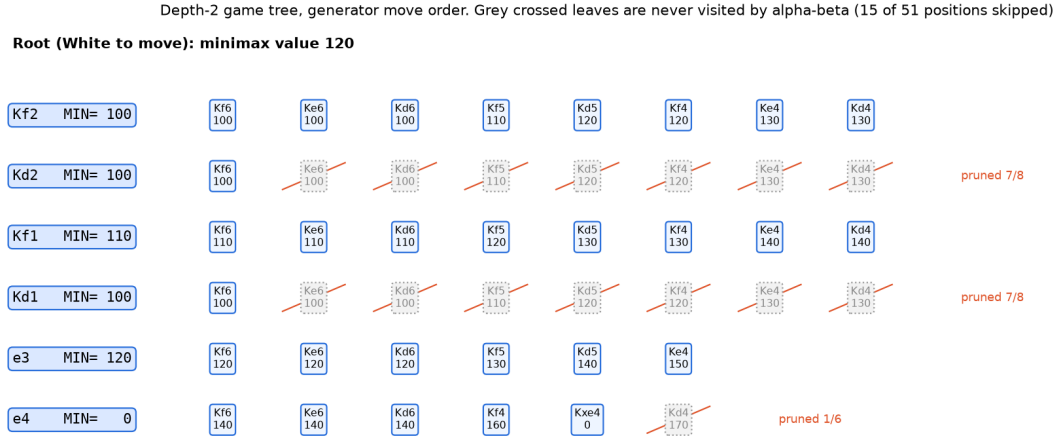


Figure 1: Depth-2 game tree. Blue boxes are MIN nodes (after White’s move) with their backed-up values. Leaves show Black’s reply and the heuristic value. Grey, crossed leaves are skipped by alpha-beta.

7 Working Cases and Edge Cases

Table 1 lists the move, value and cost of both agents at depths 1–4 in every case. Values are from White’s point of view; “mate (+)” means a forced mate for White was found.

Table 1: Case results (depth-4 minimax on E1 is omitted: about 4 million nodes, and alpha-beta gives the same value).

Case	Agent	Depth	Move	Value	Nodes	Time (ms)
W1	minimax	1	Rd8#	mate (+)	21	0
W1	alpha-beta	1	Rd8#	mate (+)	21	1
W1	minimax	2	Rd8#	mate (+)	173	4
W1	alpha-beta	2	Rd8#	mate (+)	40	2
W1	minimax	3	Rd8#	mate (+)	3,202	67
W1	alpha-beta	3	Rd8#	mate (+)	419	10
W1	minimax	4	Rd8#	mate (+)	26,207	689
W1	alpha-beta	4	Rd8#	mate (+)	780	28
W2	minimax	1	e4	-55	13	0
W2	alpha-beta	1	e4	-55	13	0
W2	minimax	2	e4	-65	164	3
W2	alpha-beta	2	e4	-65	75	2
W2	minimax	3	Nc7+	340	1,923	44
W2	alpha-beta	3	Nc7+	340	112	4
W2	minimax	4	Nc7+	320	29,745	673
W2	alpha-beta	4	Nc7+	320	222	11
E1	minimax	1	Bxc2	285	44	2
E1	alpha-beta	1	Bxc2	285	44	2
E1	minimax	2	Bxc2	615	2,263	113
E1	alpha-beta	2	Bxc2	615	140	15
E1	minimax	3	Bg4	-165	93,207	4,990
E1	alpha-beta	3	Bg4	-165	3,900	251
E1	alpha-beta	4	Bg4	390	34,023	2,588
E2	minimax	1	Kh5	520	25	1
E2	alpha-beta	1	Qe5+	520	25	1
E2	minimax	2	Qb8+	550	198	8
E2	alpha-beta	2	Qb8+	550	53	4
E2	minimax	3	Qb8+	mate (+)	4,253	100
E2	alpha-beta	3	Qb8+	mate (+)	412	14
E2	minimax	4	Qb8+	mate (+)	28,275	931
E2	alpha-beta	4	Qb8+	mate (+)	771	31

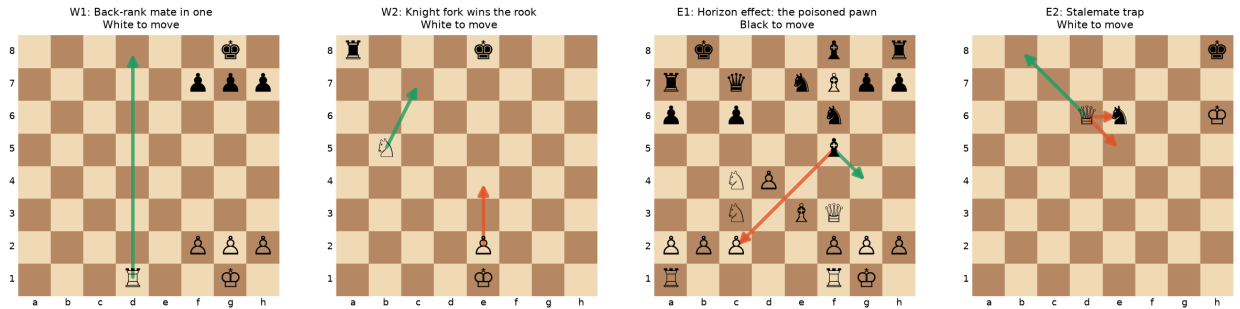


Figure 2: Case positions. Green = move chosen at the deepest search; orange = the tempting wrong move (shallow search or greedy capture).

Working case W1 – Back-rank mate in one

White: Kg1, Rd1, pawns f2 g2 h2; Black: Kg8, pawns f7 g7 h7. **Rd8#** mates because Black’s own pawns block the king. Both agents find it at every depth ≥ 1 , since the terminal test scores the position after Rd8 as a mate. The mate score 100000 – ply makes a mate-in-1 beat any material

gain. Minimax needs 3,202 nodes at depth 3 against 419 for alpha-beta. Once the mating move is searched first (it gives check, so it is ordered early), all alternatives are cut off quickly.

Working case W2 – Knight fork

White: Ke1, Nb5, pawn e2; Black: Ke8, Ra8. **Nc7+** checks the king and attacks the rook. After the forced king move, Nxa8 wins the rook. At depths 1–2 the agents cannot see the capture (it is ply 3) and play the quiet pawn move e4 with a negative value, because Black is up the exchange. From depth 3 both agents play Nc7+ with a positive value. This is the normal case: the tactic is within the horizon, both searches agree, and alpha-beta needs 112 nodes against 1,923.

Edge case E1 – The horizon effect (from a real arena game)

This position occurred in one of our agent games (Black to move). At depths 1 and 2 Black plays **...Bxc2**, winning a pawn. The refutation is **Bf4!**, which pins the black queen on c7 to the king on b8. The queen is lost *on the following moves*, beyond a 2-ply horizon. The shallow search thinks it has won a pawn (value 615 from White’s view, i.e. better for Black than the alternatives), while the depth-4 search evaluates **...Bxc2** at +850 and prefers **...Bg4** (+390). This is the classic *horizon effect*: a depth-limited agent cannot see a consequence one ply past its cut-off, and alpha-beta suffers exactly as much as minimax at the same depth. The fix is to search deeper, which alpha-beta makes affordable (depth 4 here takes alpha-beta 34,023 nodes, fewer than minimax needs at depth 3), or to add a *quiescence search* that keeps searching captures past the cut-off.

The table also shows an *odd/even effect*. At odd depths Black moves last and faces no reply, so values swing in Black’s favour (−165 at depth 3 vs. +390 at depth 4 for the same best move).

Edge case E2 – Stalemate trap

White: Kh6, Qd6; Black: Kh8, Ne6. **Qxe6** wins a knight, and a greedy agent that counts material plays it, but it leaves Black with no legal move and not in check: *stalemate*, a draw. Our agents score stalemate as 0, so at depth 1 Qxe6 is worth 0 and they choose another move. From depth 3 they find a *forced mate* beginning with **Qb8+**. Note the tie at depth 1: minimax plays Kh5 and alpha-beta plays Qe5+, both valued 520. With equal values the move depends only on the order in which moves are examined. This is why equal-depth games between the two agents can differ (Section 9).

Edge case E3 – Worst-case move ordering

Alpha-beta’s savings are not guaranteed. Table 2 runs it with three orderings. With MVV-LVA, good moves come first and refutations are found immediately. With *reversed* MVV-LVA ordering (quiet, bad moves first) cut-offs come late: in the Italian opening at depth 4, alpha-beta then needs 403,780 nodes instead of 7,372. The value returned is identical in all three cases. Only the effort changes, which is the practical meaning of the $O(b^{d/2})$ best case vs. $O(b^d)$ worst case.

8 Complexity Analysis

Let b be the branching factor (legal moves per position) and d the search depth in plies.

- **Minimax**: visits every node of the tree, so time $O(b^d)$. It is depth-first, so memory is $O(bd)$ (the current path plus the move list at each level).
- **Alpha-beta**: same memory $O(bd)$. Time depends on the ordering. In the worst case nothing is pruned: $O(b^d)$. With perfect ordering Knuth and Moore showed it examines $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$ leaves, i.e. $O(b^{d/2})$. The effective branching factor drops from b to $\sqrt{b}$, so *in the same time alpha-beta can search about twice as deep*.
- **In chess**: $b \approx 20$ at the start and ≈ 35 in the middlegame. At $d = 4$, minimax faces $35^4 \approx 1.5$ million leaves while the alpha-beta minimum is $2 \cdot 35^2 \approx 2,450$.

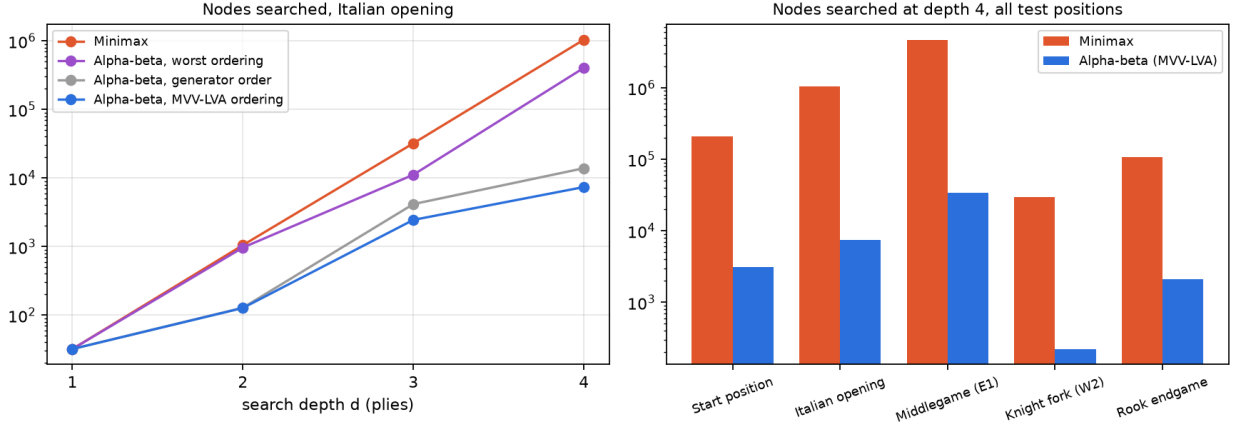


Figure 3: Left: nodes searched as depth grows (log scale, Italian opening). Right: all test positions at depth 4.

Table 2: Nodes searched (root included). Worst/None/MVV = alpha-beta with reversed, generator and MVV-LVA ordering; “Saved” compares MVV alpha-beta with minimax. Every row returned the same value for all four searches.

Position	d	Move	Value	Minimax	AB worst	AB none	AB MVV	Saved
Start position	1	Nf3	50	21	21	21	21	0.0%
Start position	2	Nf3	0	421	193	84	84	80.0%
Start position	3	Nf3	50	9,323	2,136	778	777	91.7%
Start position	4	Nf3	0	206,604	25,534	3,155	3,109	98.5%
Italian opening	1	Nf6	-30	32	32	32	32	0.0%
Italian opening	2	Nf6	100	1,048	966	128	127	87.9%
Italian opening	3	Qf6	-145	32,072	11,153	4,160	2,447	92.4%
Italian opening	4	Nf6	110	1,040,115	403,780	13,879	7,372	99.3%
Middlegame (E1)	1	Bxc2	285	44	44	44	44	0.0%
Middlegame (E1)	2	Bxc2	615	2,263	2,238	894	140	93.8%
Middlegame (E1)	3	Bg4	-165	93,207	20,781	16,107	3,900	95.8%
Middlegame (E1)	4	Bg4	390	4,701,529	764,022	184,641	34,023	99.3%
Knight fork (W2)	1	e4	-55	13	13	13	13	0.0%
Knight fork (W2)	2	e4	-65	164	39	74	75	54.3%
Knight fork (W2)	3	Nc7+	340	1,923	416	186	112	94.2%
Knight fork (W2)	4	Nc7+	320	29,745	1,720	444	222	99.3%
Rook endgame	1	Rd7+	-20	20	20	20	20	0.0%
Rook endgame	2	Rg5	-40	358	310	71	85	76.3%
Rook endgame	3	Rd2	0	6,190	1,768	741	546	91.2%
Rook endgame	4	Rd2	-10	108,368	11,457	3,161	2,124	98.0%

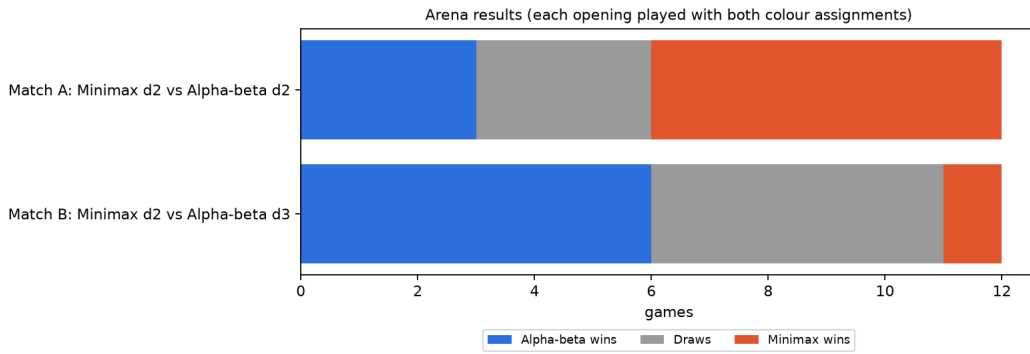
Relating the measurements to theory. From the start position ($b = 20$), minimax grows by a factor of about 20–22 per ply ($421 \rightarrow 9,323 \rightarrow 206,604$), which is exactly b . Alpha-beta grows by a factor of about 4–10 per ply ($84 \rightarrow 777 \rightarrow 3,109$), close to $\sqrt{b} \approx 4.5$ at even steps. That is the $O(b^{d/2})$ behaviour. Its depth-4 count is 3,109, compared with a theoretical minimum of $20^2 + 20^2 - 1 = 799$ leaves, so our simple ordering is good but not perfect. The middlegame position ($b = 44$) shows the gap most clearly: 4,701,529 vs. 34,023 nodes at depth 4.

9 Arena: the Agents Playing Each Other

Each match uses 6 openings (Italian, Sicilian, Queen’s Gambit, French, English, Caro-Kann). Each opening is played twice with the colours swapped, 12 games in total. A game is capped at 100 plies; unfinished games are adjudicated as a win if one side leads by ≥ 300 centipawns and as a draw otherwise.

Table 3: Arena summary. Score = points for Minimax – points for Alpha-beta (win 1, draw $\frac{1}{2}$). Nodes and milliseconds are averages per move.

Match	Games	MM – AB	MM nodes	AB nodes	MM ms	AB ms
A (equal depth 2)	12	7.5 – 4.5	1,381	169	72	14
B (AB depth 3)	12	3.5 – 8.5	1,206	2,214	58	133



Match A – equal depth (both depth 2). Minimax scored 7.5–4.5. This does *not* mean minimax plays better. We replayed all 636 positions from these games and asked both agents for their move. The chosen moves had the **same minimax value in 100.0% of positions**, and the agents chose the identical move in 98.7%. The few differences are tie-breaks between equally valued moves (as in E2 at depth 1), caused by alpha-beta’s different move order. Once one tie is broken differently the two games diverge, and at depth 2 the play is weak enough (many games end in checkmate after blunders) that such divergences decide games. With deterministic agents and 12 games the score is essentially noise. The important column is the cost: alpha-beta needed about $8\times$ fewer nodes per move for the same decisions.

Match B – alpha-beta spends its savings on depth (depth 3 vs. depth 2). A depth-3 minimax would need roughly 9–93 thousand nodes per move (Table 2), while the depth-3 alpha-beta agent needed about 2,200, less than twice what minimax spends at depth 2. With this extra ply the alpha-beta agent won 6 games, drew 5 and lost 1 (score 8.5–3.5). It sees the opponent’s reply to its reply, so it avoids the one-move traps that decide games at depth 2. The draws are threefold repetitions: deterministic agents with a shallow horizon tend to shuffle pieces back and forth once no tactic is visible, which is another face of the horizon effect.

Table 4: All games of Match B (MM = minimax, AB = alpha-beta; the number is the depth).

Opening	White	Black	Result	Termination	Plies
Italian	MM:2	AB:3	1/2-1/2	threefold repetition	31
Italian	AB:3	MM:2	1/2-1/2	threefold repetition	29
Sicilian	MM:2	AB:3	0-1	checkmate	58
Sicilian	AB:3	MM:2	1-0	checkmate	31
Queen’s Gambit	MM:2	AB:3	1/2-1/2	threefold repetition	71
Queen’s Gambit	AB:3	MM:2	1-0	checkmate	55
French	MM:2	AB:3	1-0	checkmate	35
French	AB:3	MM:2	1-0	adjudicated (eval +475)	100
English	MM:2	AB:3	1/2-1/2	threefold repetition	35
English	AB:3	MM:2	1-0	checkmate	61
Caro-Kann	MM:2	AB:3	0-1	checkmate	64
Caro-Kann	AB:3	MM:2	1/2-1/2	threefold repetition	67

10 Comparison and Conceptual Understanding

	Minimax	Alpha-beta (MVV-LVA ordering)
Search process	Depth-first over every move sequence to depth d	Depth-first with (α, β) window; skips moves once a refutation is found
Solution quality	Optimal w.r.t. the heuristic at depth d	<i>Identical</i> value (verified on every position and depth)
Nodes, depth 4 (5 positions)	1,217,272 on average	9,370 on average
Time complexity	$O(b^d)$ always	$O(b^{d/2})$ best, $O(b^d)$ worst
Space	$O(bd)$	$O(bd)$
Depends on move order	No	Yes (E3)
Reachable depth (≈ 1 s/move, Python)	3	4–5
Limitations	Exponential; shallow in practice; horizon effect	Still exponential; ordering-sensitive; same horizon effect at equal depth

What happens as the search space grows. Each extra ply multiplies minimax’s work by b (20–44 in our positions) and alpha-beta’s by roughly $\sqrt{b}$ with good ordering. Middlegames ($b \approx 44$ in E1) are therefore far more expensive than endgames ($b \approx 12$ –20). At depth 4 on the E1 middlegame, minimax already needs about 4 million nodes (several minutes in Python), while alpha-beta needs tens of thousands. Neither technique removes the exponential growth. Real engines add iterative deepening, transposition tables, quiescence search and better evaluations on top of alpha-beta.

Why this search technique. Chess is a deterministic, fully observable, two-player zero-sum game, which is exactly the setting in which minimax defines optimal play, so expectimax or MCTS are not needed. Alpha-beta keeps minimax’s guarantee and cuts the cost, so at the same budget it searches deeper. The arena shows that depth, not pruning itself, is what converts into playing strength.

11 Conclusion

We modelled chess as a competitive multi-agent environment and built two utility-based agents that differ only in their search technique. The experiments confirm the theory. Alpha-beta always returned the minimax value, and with MVV-LVA ordering it searched 1–2 orders of magnitude fewer nodes, growing close to $\sqrt{b}$ per ply instead of b . The edge cases show the limits: both agents suffer the horizon effect at shallow depth (E1), correct terminal scoring is essential to avoid stalemating

a won game (E2), and alpha-beta’s gain depends on move ordering (E3). In head-to-head play the agents are equally strong at equal depth. Given comparable computation, however, the alpha-beta agent searches one ply deeper and wins. Natural extensions are iterative deepening with a time limit, quiescence search against the horizon effect, and a richer evaluation (mobility, king safety, pawn structure).

References

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021, Ch. 5 (Adversarial Search).
- [2] C. E. Shannon, “Programming a Computer for Playing Chess,” *Philosophical Magazine*, 41(314), 1950.
- [3] D. E. Knuth and R. W. Moore, “An Analysis of Alpha-Beta Pruning,” *Artificial Intelligence*, 6(4), 1975.
- [4] T. Michniewski, “Simplified Evaluation Function,” Chess Programming Wiki.
- [5] N. Fiekas, `python-chess`: a chess library for Python, <https://python-chess.readthedocs.io>.

A Reproducing the Results

- `python code/experiments.py` regenerates every table and figure in this report.
- `python code/chess_agents.py --white minimax:2 --black alphabeta:3` plays and prints a full game.
- See `README.md` for all options.